

Report Challenge: Learning to Walk with PPO

Student Number: K51843919

Student Name: Lukas Steu

Setup

I ran the challenge in Google Colab with a Python 3 runtime and a T4 GPU. I also mounted Google Drive, because otherwise the checkpoints and ONNX files would be lost when the runtime disconnects. I first tried to keep the original notebook structure as much as possible and mainly changed the parts that were necessary. Later I added some extra cells for saving checkpoints, exporting ONNX files and evaluating different models. I also installed onnxscript, because I had some problems with exporting the model to ONNX and loading it again with onnx2pytorch.

The environment used was BipedalWalker-v3 with hardcore=False. One important change was that I increased the maximum episode length from 400 to 2000 steps, because the official evaluation script also allows up to 2000 steps. This made training slower, but it made the training setup more similar to the final challenge evaluation.

Changes of the Code

At first, I implemented the missing actor and critic networks. Both networks are simple feed-forward networks with two hidden layers. I did not use a deep architecture, because the assignment hints said that shallow networks work better for this task. The actor returns `mu_logits` and `sigma_logits`, which are then converted in the same way as in the evaluation script. Later I changed the sigma part of the actor. Instead of predicting sigma from every state, I used one global trainable sigma parameter. This made the policy more stable.

The buffer was also changed quite a lot. The original buffer only stored states, actions, rewards, next states and log probabilities. I added done and advantage to the transitions, because I implemented GAE. GAE stands for Generalized Advantage Estimation and is used to get a more stable advantage estimate for PPO. This was one of the most important changes, because the earlier versions either learned very slowly or got stuck at low rewards. The dataloader also uses `drop_last=True`, because I had problems with very small batches causing NaN values.

The PPO training loop was implemented in the Agent class. It calculates the policy ratio, clips it with `eps_clip`, computes the policy loss, value loss and entropy loss, and then updates the network. I also added advantage normalization, gradient clipping and a check that skips updates if the loss becomes NaN or infinite. The critic was trained with a higher learning rate than the actor, because the assignment specifically mentioned that the critic should learn faster.

The Runner was completed so it actually collects episodes, stores transitions, fills the buffer, trains the agent and logs the results. I changed the checkpointing so that the best checkpoint is saved based on the average reward of the last 10 episodes instead of just one single episode. This was important because one lucky episode does not always mean that the model is actually good. I also added random seeds during training to make the agent see more different situations. Additionally, I added a small fall penalty. This penalty is only used during training when the agent falls before

reaching the maximum episode length. The idea was to make early falling less attractive and reduce the negative reward drops that happened when the walker fell very early.

Another large part was fixing the ONNX export. The original export did not work reliably for me. I had CPU/CUDA mismatch errors and also errors with onnx2pytorch. Therefore, I forced the ONNX export to run fully on CPU. I also added an ActorONNXWrapper, which makes both outputs depend on the input tensor, so that onnx2pytorch does not treat them as constants. The final export uses `opset_version=18`, `do_constant_folding=False`, `dynamo=False`, and the output names `mu` and `sigma`.

Finally, I added evaluation cells. One evaluation cell runs the exported ONNX model almost like the official challenge script over 50 episodes. I also added a deterministic mean-policy evaluation to check whether the bad episodes came from sampling or from the policy itself. At the end, I evaluated several checkpoints and selected the model that had the best average score and not only the highest single reward.

Hyperparameters & Results

For the final fine-tuning run I used 1500 episodes resuming from an earlier checkpoint (iteration 13), 3 PPO epochs, hidden size 128, learning rate $3e-5$, discount 0.99, reward scaling with reward / 10, batch size 64, `min_transitions=1024`, buffer capacity 2048, `eps_clip=0.1`, loss scales (0.7, 0.001), GAE with `lambda=0.95`, and a fall penalty of 5.0. The final model was exported as `final_model.onnx`.

The best local ONNX evaluation result was approximately:

Average Reward: 263.95395

Standard Deviation: 81.24121

Minimum Reward: -81.16481

Maximum Reward: 309.29337

The model can walk quite well (it seems to jump with one foot forward and uses the other to stabilize the jump) in many episodes and often reaches rewards around 180 to 220. However, it still sometimes falls early, which creates large negative outliers. Because of that, I did not only look at the best single episode, but mainly at the 50-episode average and how many bad episodes occurred.